



content-engine

GUIDE

From Illustrator to Web Animations with Claude Code

Turn hand-drawn Adobe Illustrator characters into editable, animated React components you can refine in plain language and drop into a website.

content-engine (Will Vessels) · 2026-06-27

For: Designers and makers who have an Illustrator file and want animated web characters without hand-coding every frame

Learning objectives

- Recognize why an editable file structure, not a one-off export, is what makes design work repeatable.
- Prepare an Adobe Illustrator file by naming its layers and exporting it as SVG so an AI can work with it.
- Install the two building blocks this workflow relies on: the Remotion library and the UI/UX Pro skill.
- Set up Claude Code in an editor so it can read your project and change files in place.
- Refine animated characters in plain language and move a finished one into a React website.

What this builds

A friend needed a website for a storyboard he had drawn, and he wanted the characters from it to come alive. As you scroll the page, the story unfolds, and scattered through it are little characters you can jump into, each with its own animations and micro expressions, almost reacting to one another. He also wanted a full video on the site that walks through every character in its animated form.

There was a catch. He was doing this for a professor who loves design and did not believe AI could generate this kind of animation in a way that stayed clean, organized, and editable, built from the original Adobe Illustrator files. This guide walks through how that challenge was met: hand-drawn art becomes animated web characters you can keep editing, all driven from a single folder.



The finished site: a hand-drawn character, now animated, running live in the browser.

NOTE

The goal is not a single exported animation. It is a folder you can keep working in, where every character stays editable long after the first version is done.

Why a file structure, not an app

The big idea behind this approach is simple to say: you do not need a pile of separate apps, you need a good file structure. Once the project lives as an organized folder of files, that folder becomes the thing you edit. You can build and refine videos, websites, or characters without opening Illustrator or any other single-purpose tool, because the work itself is just files you can change.

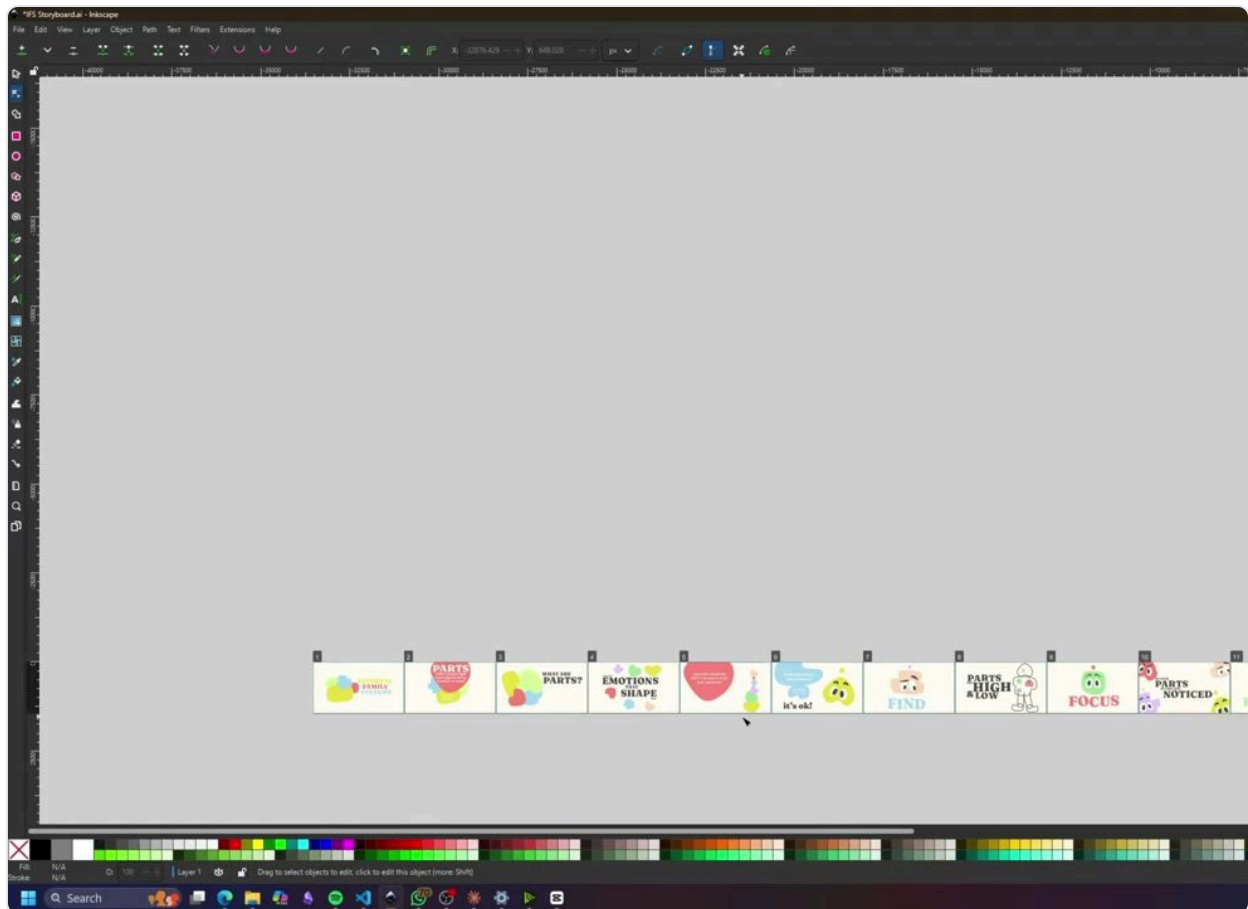
That shift matters because it changes how you make edits. Instead of clicking through menus and panels, you describe what you want in plain language and let the AI make the change in the right file. The design knowledge still matters, and the original Illustrator art is still where everything starts. What changes is that the result stays open and editable rather than locked inside one program.

TIP

Keep this framing in mind as you read the steps: every step below exists to turn the drawing into editable files and then give an AI a clean way to work on them.

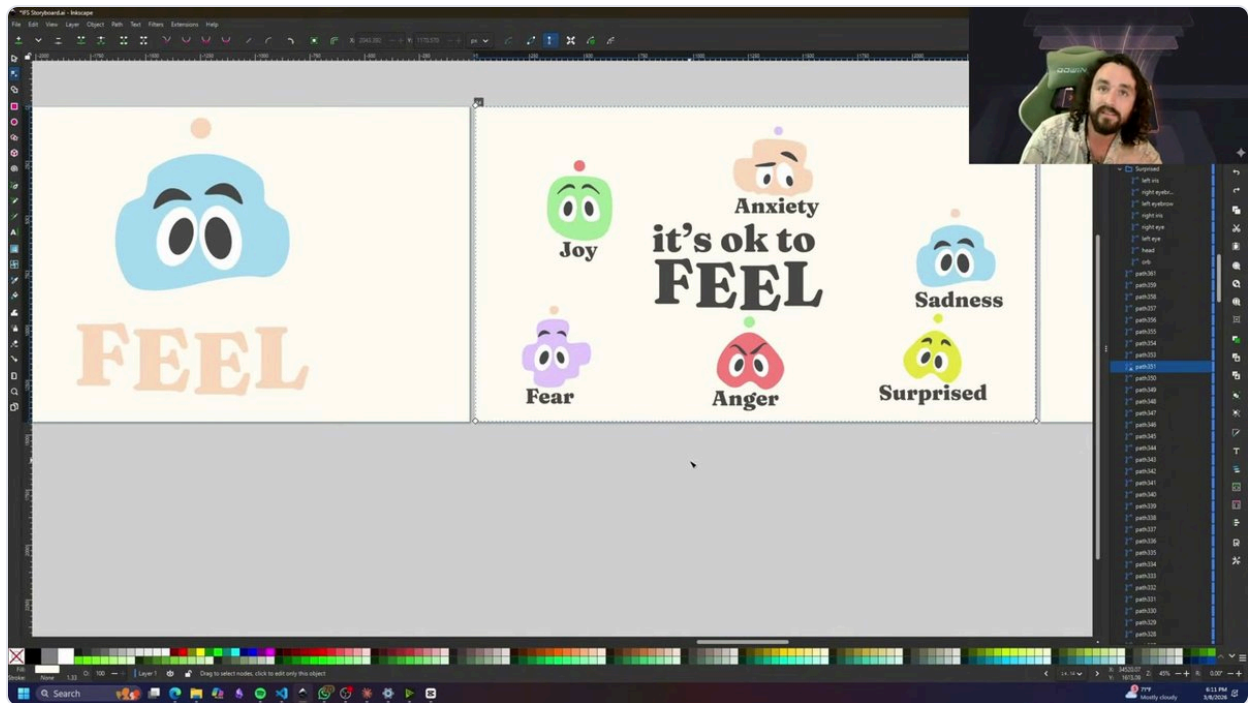
Start with the Illustrator file

Everything starts with the original Adobe Illustrator storyboard. Each panel was hand designed and placed by the artist, so the source is real design work, not a template. The first job is to get that artwork out of Illustrator and into a form the AI can read directly.



The hand-drawn storyboard: each panel of the story is its own designed frame.

Before exporting, the layers get clear, consistent names. Working together, the artist relabeled the pieces of each character: left iris, right iris, left eye, right eye, eyebrow left, eyebrow right, and so on. The reason is that the AI looks for those named parts so it knows exactly which piece is which. It can take screenshots to figure things out, but naming the layers up front skips that step and lets it go straight to the right element.



Named layers on the right (left iris, right eye, eyebrow, head) make each character part easy to find.

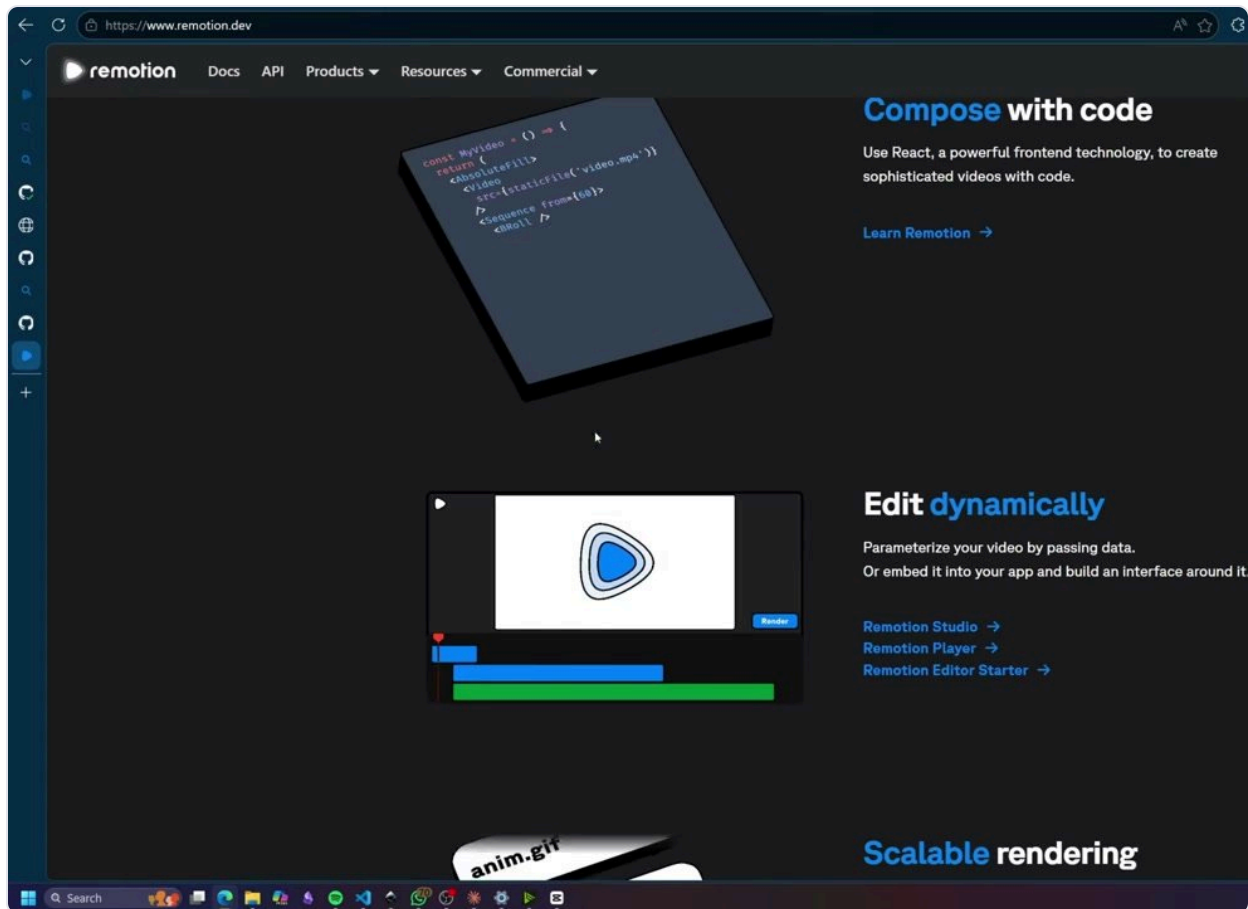
With the parts named, the whole storyboard is exported as a single SVG file. An SVG is essentially a big structured file much like an HTML file, so it is easy for the AI to open and work with directly. That one export becomes the shared starting point for everything that follows.

TIP

Naming layers is the small bit of human setup that pays off everywhere later. Consistent names like left iris and right eyebrow let you ask for an edit by name instead of pointing at pixels.

Add Remotion for the animations

This workflow leans on two building blocks you grab from GitHub. The first is **Remotion**, a library that lets you create animations programmatically with React. Instead of drawing each frame by hand, you describe animations in JavaScript and React code, which is exactly the kind of work AI is good at writing. Remotion can go well beyond small clips and build entire scenes in code.

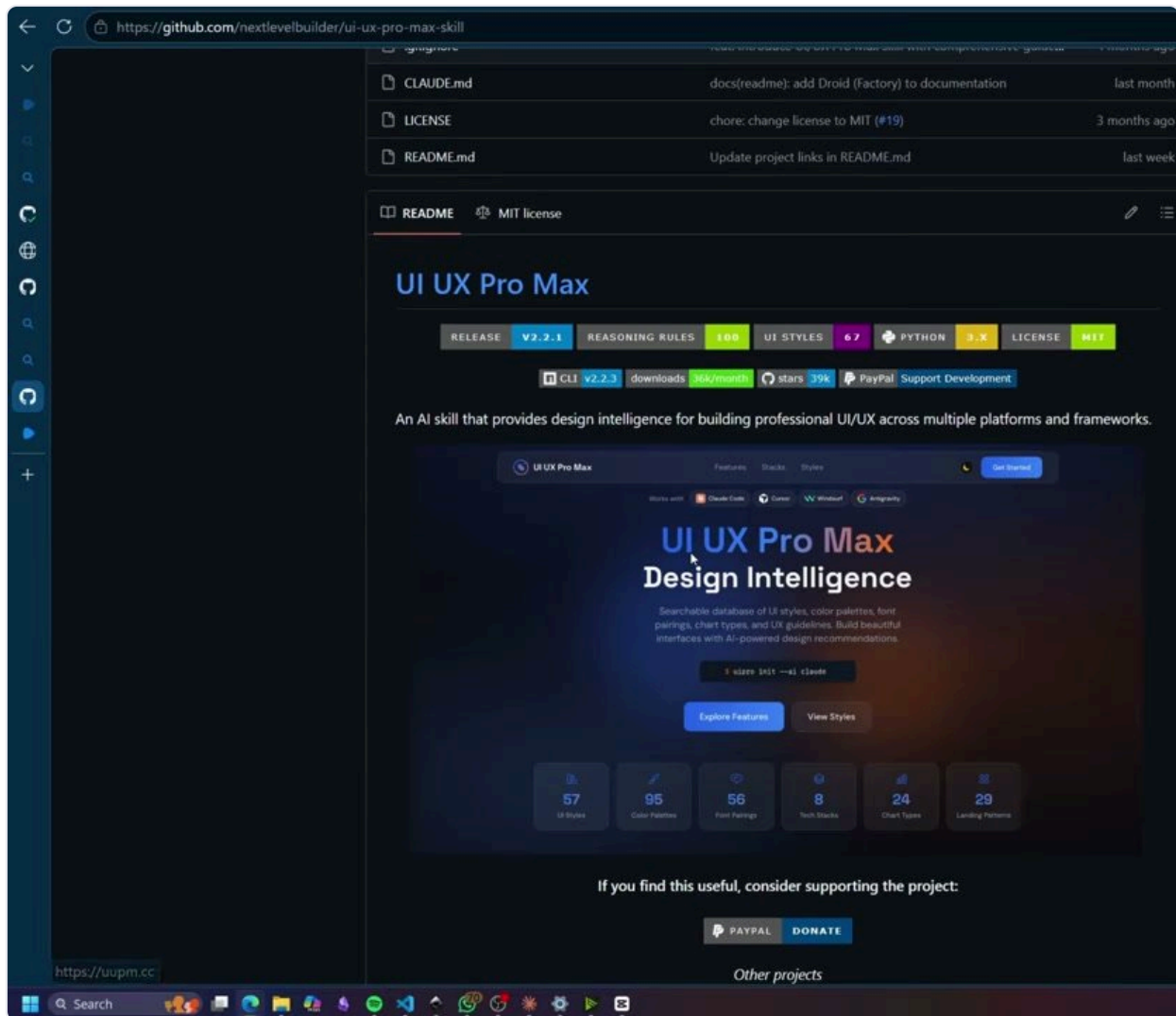


Remotion lets you compose video and animation with React code instead of a timeline.

Remotion is the engine that animated the storyboard characters. Because the animations are code, they can be generated, tweaked, and regenerated, which is what lets the rest of this workflow stay flexible.

Add the UI/UX Pro skill for clean design

The second building block is a skill called **UI/UX Pro**. It gives the AI design intelligence for building a professional front end: how a site should look, which styles, colors, and patterns fit. Without it, you tend to prompt over and over to get close to a clean design. The skill carries that knowledge built in, so the result looks intentional instead of hastily put together.



The UI/UX Pro skill adds design know-how so the front end looks polished, not improvised.

NOTE

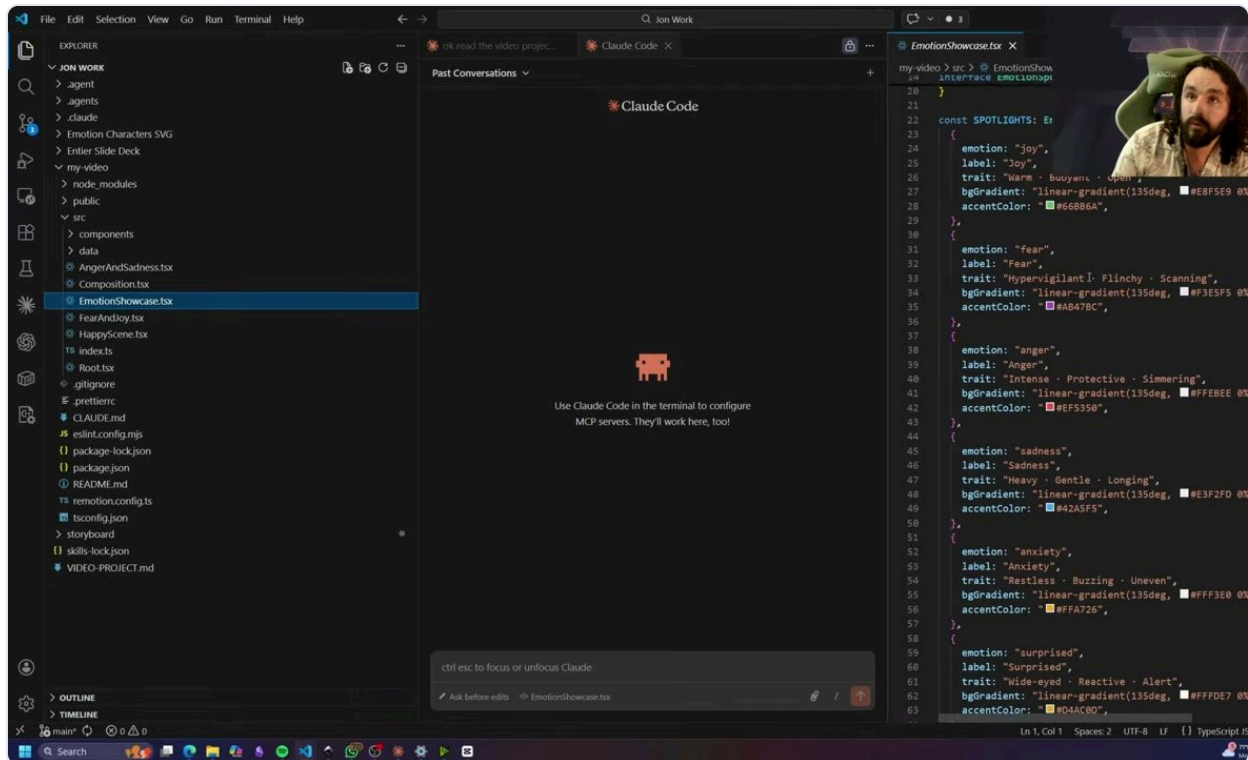
Both pieces, Remotion and the UI/UX Pro skill, are installed from GitHub and are treated as requirements for this workflow. One handles the animation, the other handles the look.

Work in an editor with Claude Code

With the pieces in place, the work happens in an editor with Claude Code running alongside the files. A code editor like VS Code is a comfortable home for this, because you can click a character, see it right there, and talk to the AI about the project while it reads and edits the

actual files. The editor is a preference, not a requirement; the same Claude Code works from the terminal too.

Building the characters was mostly a long back-and-forth conversation: take these individual SVGs and make animations from them, try a design, say what worked and what did not, and let the AI edit. Behind the scenes, it is pulling each SVG into Remotion components, so the drawings become real, editable building blocks of the project.



The project as a folder: characters, scenes, and data as files Claude Code can read and edit in place.

Look at the file tree and you can see the shape of the project: a folder of components for the characters, a data file for their details, scenes that assemble them, and the storyboard. Each character keeps its own settings, such as its head and colors, when it blinks, and how fast its irises move. Because all of that lives in plain files, it stays editable.

TIP

Add a project file named **CLAUDE.md** so the AI reads it automatically. Point it at where things live, such as which folders hold which parts and where the original SVG is, so the AI starts with that context instead of re-reading the whole codebase every time.

Refine in plain language, then move it to the site

Once the project is a set of files, editing becomes a conversation. You can say something like, I do not like the eyes on the sad character, what can I do, and the AI does not have to rework the whole animation. It finds the specific component for that character, adjusts just the values that matter, the iris speed or the movement for that emotion, and leaves the rest of the code alone. You can hand-edit any of it yourself, or let the AI make the small, refined change.

This is the difference from tools where one change forces you to redo an entire video or risks breaking half of what you built. Here the edit is surgical. When a character looks right in its scene, you can take that finished character and ask to add it into the React website, which is exactly how the storyboard site gained its animated characters.

NOTE

Plain-language editing does not mean giving up fine control. You capture your intent in words while every small detail stays adjustable, by you or by the AI.

What you can take from this

The path is repeatable: start with real Illustrator artwork, name its layers and export it as SVG, bring in Remotion for the animation and the UI/UX Pro skill for the design, and work in an editor with Claude Code so it can read and change the files directly. From there you refine characters in plain language and move the ones you like into the website.

What used to take weeks by hand came together in a few hours, but the design judgment and the original artwork still did the heavy lifting. The real takeaway is the structure: when your work lives as editable files, you can do all of it inside a single folder with an AI, rather than bouncing between separate apps.

NOTE

Some edits are still faster in Illustrator, and some pieces are best left alone once the scenes are assembled. The skill is deciding where the human belongs in the loop and where the AI can carry the work.

Tools and links

- Remotion (React animation library): remotion.dev

- UI/UX Pro skill (design intelligence for the front end): github.com
- Claude Code (the AI that reads and edits your files): claude.com
- Adobe Illustrator (the original artwork): adobe.com

Educational guide generated by content-engine from a public recording. Every claim traces to the source transcript via the provenance sidecar; the reader surface carries no timestamps or citations. · Source: 'SVG to React: Turning Illustrator Designs into Web Animations with Claude Code' (public YouTube).